



Veterinaria



Java + Spring Boot



JDBC + MySQL



Nivel inicial

Trabajo Práctico Integrador

Desarrollo de una API básica para la gestión de una veterinaria usando **Spring Boot**, **JDBC** y **MySQL**. El objetivo es que den sus primeros pasos en el framework, comiencen a familiarizarse con la organización general de un proyecto y practiquen la conexión con una base de datos relacional.



MODALIDAD

Individual o en parejas

ENTREGA

Proyecto funcional +
script SQL + evidencias
de prueba

CONDICIÓN

No usar JPA ni Hibernate



Render MVC / DAO
/ JDBC endpoints REST





Consigna general

Una veterinaria necesita un sistema simple para registrar clientes, mascotas, veterinarios y turnos. Ustedes deberán desarrollar una aplicación en Spring Boot que permita guardar, consultar, modificar y eliminar información básica del sistema.

Importante: este trabajo está pensado para estudiantes que recién comienzan con Spring Boot. Se prioriza que el proyecto esté **claro, ordenado y funcionando** antes que agregar funciones complejas.



Objetivos del trabajo

- Crear un proyecto desde **Spring Initializr**.
- Comprender la estructura general de una app Spring Boot.
- Conectar la aplicación a una base de datos MySQL.
- Realizar operaciones CRUD usando **JDBC**.
- Organizar el código con una lógica tipo **MVC + DAO**.
- Exponer endpoints REST para probar el sistema con navegador o Postman.



Tecnologías obligatorias

Java

Versión 17 o superior

Spring Boot

Proyecto creado con Initializr

Spring Web

Para exponer endpoints

JDBC API

Para acceso a datos

MySQL Driver

Para la conexión con MySQL

MySQL

Motor de base de datos



Configuración inicial del proyecto

- Ingresar a [Spring Initializr](#) .
- Crear un proyecto **Maven** con lenguaje **Java**.

- Completar los datos básicos del proyecto (grupo, artefacto, nombre, paquete).
- Agregar las dependencias: Spring Web , JDBC API y MySQL Driver .
- Importar el proyecto en el IDE y comprobar que inicia correctamente.

Escenario

La veterinaria "Patitas Felices" quiere digitalizar la información más importante de su atención diaria. El sistema deberá registrar a los clientes, sus mascotas, los veterinarios disponibles y los turnos que vinculan cada atención.

Lo mínimo que debe poder hacer

- Guardar nuevos registros.
- Listar los datos cargados.
- Buscar por ID.
- Modificar registros existentes.
- Eliminar registros cuando sea necesario.

Aclaración de modelado

En este trabajo, **cliente** y **dueño** se consideran la misma entidad. Por eso se utilizará una sola tabla para representar a la persona responsable de la mascota.

Sugerencia didáctica

Empiecen primero por que el proyecto arranque, luego configuren la conexión, después creen las tablas y recién entonces avancen con DAO, modelos y controladores.



Modelo de datos solicitado

La base de datos deberá contener **4 tablas**: **cliente**, **mascota**, **veterinario** y **turno**. La tabla **turno** funcionará como tabla intermedia entre **cliente** y **veterinario**, y además deberá indicar qué mascota será atendida.

Tabla: cliente

- **id_cliente** (PK)
- **nombre**
- **apellido**
- **telefono**
- **email**
- **direccion**
- **activo**

Representa al dueño o responsable de una o más mascotas.

Tabla: mascota

- **id_mascota** (PK)
- **nombre**
- **especie**
- **raza**
- **edad**
- **peso**
- **id_cliente** (FK)

Cada mascota debe quedar vinculada a su cliente/dueño.

Tabla: veterinario

- **id_veterinario** (PK)
- **nombre**
- **apellido**
- **matricula**
- **especialidad**
- **telefono**
- **email**

Representa a los profesionales que atenderán los turnos.

Tabla: turno

- **id_turno** (PK)
- **fecha**
- **hora**
- **motivo**
- **estado** (pendiente / confirmado / cancelado / atendido)
- **id_cliente** (FK)
- **id_veterinario** (FK)
- **id_mascota** (FK)

Conecta al cliente con el veterinario y deja registrada la mascota que será atendida.

Cliente 1 → N Mascotas

Cliente 1 → N Turnos

Veterinario 1 → N Turnos

Mascota 1 → N Turnos



Organización sugerida del proyecto

No es obligatorio que todos los grupos lo hagan exactamente igual, pero se recomienda una estructura simple y ordenada:

1

Paquete model

Clases Java que representen las tablas: `Cliente` , `Mascota` , `Veterinario` y `Turno` .

2

Paquete dao

Clases encargadas de acceder a la base de datos usando JDBC. Aquí se escriben las consultas SQL.

3

Paquete controller

Controladores REST para exponer endpoints y recibir las solicitudes del cliente.

4

Paquete config o util

Opcional. Puede contener configuración o clases auxiliares si el grupo lo considera necesario.



Requerimientos funcionales mínimos

1. Crear la base de datos en MySQL con sus 4 tablas y relaciones.
2. Configurar el archivo `application.properties` para conectarse correctamente a la base.
3. Crear las clases modelo correspondientes a cada tabla.
4. Implementar al menos un DAO por cada entidad.
5. Implementar controladores REST para consultar y modificar los datos.
6. Probar el funcionamiento con navegador y/o Postman.
7. Dejar evidencia de que la aplicación inserta, lista, modifica y elimina datos.

Condición obligatoria: el acceso a datos debe resolverse con **JDBC**. No se permite resolver este trabajo con JPA, Hibernate ni repositorios automáticos de Spring Data.

Endpoints sugeridos

Estos endpoints son una guía mínima. Pueden agregar otros si desean ampliar el trabajo.

Método	Ruta sugerida	Descripción	Obligatorio
GET	/clientes	Listar todos los clientes	Sí
GET	/mascotas	Listar todas las mascotas	Sí
GET	/veterinarios	Listar todos los veterinarios	Sí
GET	/turnos	Listar todos los turnos	Sí
GET	/turnos/{id}	Buscar un turno por ID	Sí
POST	/turnos	Crear un nuevo turno	Sí
PUT	/turnos/{id}	Modificar un turno existente	Sí
DELETE	/turnos/{id}	Eliminar un turno	Sí
GET	/mascotas/cliente/{idCliente}	Listar mascotas de un cliente	Deseable
GET	/turnos/veterinario/{idVeterinario}	Listar turnos de un veterinario	Deseable

Etapas recomendadas para resolverlo

1

Crear el proyecto y hacerlo arrancar

Primero verifiquen que la aplicación se ejecuta sin errores antes de conectarla a la base.

2

Diseñar la base de datos

Armen el script SQL, creen las tablas, definan claves primarias y foráneas y carguen algunos datos de prueba.

3

Configurar la conexión

Completen el archivo de propiedades con URL, usuario, contraseña y nombre del driver.

4

Desarrollar modelos y DAO

Construyan las clases Java y luego las consultas necesarias para insertar, listar, modificar y borrar.

5

Crear controladores REST

Expongan rutas claras para que los datos puedan probarse fácilmente desde Postman o desde el navegador.

6

Probar y corregir

Antes de entregar, asegúrense de que todas las rutas obligatorias respondan correctamente y que no haya errores de conexión.



Entregables

1. Proyecto completo

Carpeta o archivo comprimido con el código fuente del proyecto Spring Boot.

2. Script SQL

Archivo con la creación de la base de datos, tablas y, si quieren, algunos inserts de prueba.

3. Evidencias de prueba

Capturas o colección de Postman mostrando que funciona al menos el CRUD de turnos y los listados generales.

4. Breve explicación

Texto corto indicando cómo ejecutaron el proyecto y cómo probaron los endpoints.



Criterios de evaluación

Estructura del proyecto

20%

Orden de paquetes, nombres claros, prolijidad general.

Conexión con la base de datos

20%

Configuración correcta y funcionamiento sin errores.

Uso de JDBC

20%

Consultas SQL bien resueltas desde DAO.

Endpoints y funcionamiento

25%

Alta, baja, modificación y consulta de datos.

Presentación y explicación

15%

Entrega clara, evidencias y comprensión básica del trabajo realizado.

Desafío extra (opcional): agregar una consulta que devuelva cada turno junto con el nombre del cliente, el nombre de la mascota y el nombre del veterinario, usando `JOIN` entre tablas.

Este archivo está preparado para verse bien en pantalla y también para exportarse como PDF desde el navegador.